

Therefore, you can write code to check the value of `__name__` and execute some test code only if a module is running as the main program.

I'll use the `Deck` class as an example. At the end of *Deck.py*, after the code of the class, I've added this code to create an instance of the `Deck` class and print out the cards that it creates:

```
--- snip code of the Deck class ---
if __name__ == '__main__':
    # Main code to test the Deck class

    import pygame

    # Constants
    WINDOW_WIDTH = 100
    WINDOW_HEIGHT = 100

    pygame.init()
    window = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))

    oDeck = Deck(window)
    for i in range(1, 53):
        oCard = oDeck.getCard()
        print('Name: ', oCard.getName(), ' Value:', oCard.getValue())
```

This checks if the *Deck.py* file is running as the main program. In the typical case where the `Deck` class is imported by some other module, the value of `__name__` will be `'Deck'`, so this code does nothing. But if we run *Deck.py* as the main program, for testing purposes only, Python sets the value of `__name__` to `'__main__'` and this test code runs.

In the test code, we build a minimal pygame program that creates an instance of the `Deck` class, then prints out the name and value of all 52 cards. The output of running *Deck.py* as the main program looks like this in the shell or console window:

```
Name:  4 of Spades   Value:  4
Name:  4 of Diamonds Value:  4
Name:  Jack of Hearts Value: 11
Name:  8 of Spades   Value:  8
Name:  10 of Diamonds Value: 10
Name:  3 of Clubs    Value:  3
```